

React

Routing, Forms, asynchronous computation

Syllabus

- Routing
 - Meaning
 - `react-router-dom` module
 - Redirection: `navigate()`, `<NavLink>`
 - Routing parameters on the path
 - Routing parameters in the query string
- Forms:
 - Example: `<addStudentComponent>`
 - Using local hooks
- Service with a link to the controller API
 - `fetch()` method
 - `async/await` operators, `Promise<T>`
 - Hooks: `useLocation()`, `useEffect()`
 - `json-server` tool

Routing

Meaning

`react-router-dom` module

Redirection: `navigate()`, `<NavLink>`

Routing parameters on the path

Routing parameters in the query string

SPA and CSR

- In a simple case, a SPA (Single Page Application) application is entirely under a single URL path. Furthermore, the component layout remains constant throughout the application's lifespan.
- However, in most cases, we'd like components to be interchangeable. For example, in the left panel, we'd like to have a list of students at one time, and courses or teachers at another. Instead of embedding all the logic in the complex code of the left panel, it would be clearer to create different components (`<StudentListComponent>`, `<CourseListComponent>`, `<TeacherListComponent>`, etc.) and simply insert them in place of the left panel. Similar changes should apply to the right panel.
- It would also be useful to be able to link a URL directly to the student list, or even edit a specific student, as was the case with the MPA (Multi Page Application).
- Of course, you can do this yourself, but there are modules that automate these elements of SPA application operation. One such module is `react-router-dom`.

React Routers

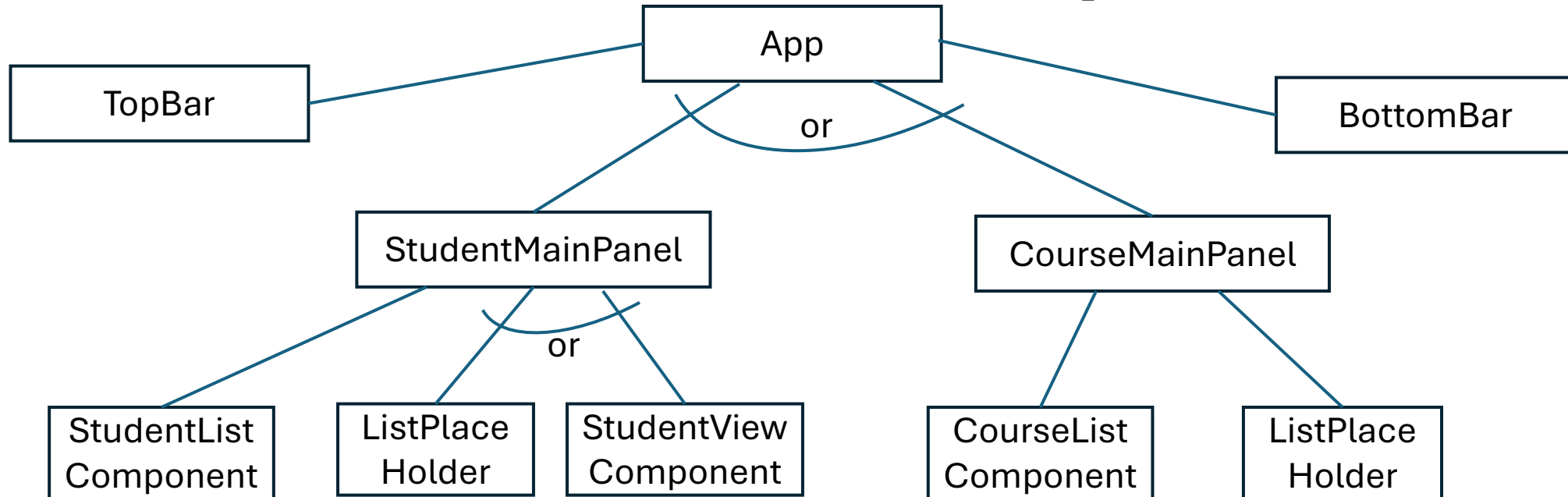
- We'll choose the basic router version for React to understand routing principles.
- Installation (in the project folder):
 - `npm install react-router-dom`
- Installation does not affect the project structure.
 - It only changes the `package.json` and `package-lock.js` files.
 - Of course, the `/node_modules` subfolder will be updated.
- Choosing the more advanced React Router v7 router option during project creation will create files and classes that simplify using the local router and hide many implementation details.
 - Once you understand routing principles, it's worth starting to use this option.
- Note: There are three module packages with similar names from the same author:
 - `react-router`: Contains path matching logic, but does not have any browser-specific elements (such as links or buttons).
 - `react-router-dom`: This is the version for web (browser) applications. It includes everything in `react-router`, plus browser-specific components like `<BrowserRouter>`, `<a>` (wrapped in `<Link>`), and back button support.
 - `react-router-native`: This is the version for mobile apps built with React Native.

Project assumptions regarding routing

- The graphical layout and general component structure tree remain the same.
- The `<TopBar>` element should display a menu with the options **students**, **courses**, and **about**.
- If you select **about** instead of `<MainPanel>`, an `<AboutComponent>` component with the selected static information will appear.
- If you select **students**, a list of students will appear in the left panel, and the behavior will continue as before.
 - However, the associated URL should be `/students`.
 - If a student (with ID=5) is selected, it will also be associated with the path `/students/view/5`.
- If you select **courses**, a list of courses appears in the left panel, and a placeholder in the right panel.
 - while the associated URL should be `/courses`
 - This is just to simplify the demonstration of routing.
- Linking URL paths to a specific application situation means that, for example, selecting a student from the list with the mouse will also change the URL path displayed in the browser, in addition to displaying that student's data. Typing the same URL path into the URL frame will also have the same effect.

Conclusions from the assumptions

- Conclusion: I need two types of main panels: a `<StudentMainPanel>`, which will contain the code from the previous lecture's `<MainPanel>`, and a `<CourseMainPanel>` for similar functionality on the course list.
 - The generic `<MainPanel>` is no longer needed. Similarly, specialized versions of the left and right panels will be created.
- Additionally, the right student panel will be separated into `<ListPlaceholder>` and `<StudentViewComponent>` components.



Layout variants

- The figures show that certain elements are interchangeable, at different levels.
- We'll use **Nested Routing** here.
- Since the components are now specialized, some for student management, others for course management, it's logical to create a folder structure for this situation (this is an example of **Feature-Based Architecture**):
 - `/components/` - general-purpose components
 - `/components/layout/` - framework components
 - `/features/students/` - student components
 - `/features/courses/` - course components
 - `/common/` - common, auxiliary components.
- To use routing capabilities, they must, of course, be imported. For example:
 - `import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';`
- Since URL navigation will determine the application's state, most of the previous **hooks** will **be replaced** with **common variables**.

Changes to the `main.tsx` file

- To use routing, place the `<BrowserRouter>` element as high as possible in the virtual DOM tree hierarchy.
 - There can **only** be **one** such element.
 - All other elements like `<Routes>`, `<Route>`, `<NavLink>`, or the `useNavigate()` hook must be within this element.
 - Therefore, it's best to insert it directly into `main.tsx`.

```
src > main.tsx
1  import { StrictMode } from 'react'
2  import { createRoot } from 'react-dom/client'
3  import './index.css'
4  import App from './App.tsx'
5  import { BrowserRouter } from 'react-router-dom';
6
7  createRoot(document.getElementById('root')!).render(
8    <StrictMode>
9      <BrowserRouter>
10     <App />
11   </BrowserRouter>
12 </StrictMode>,
13 )
```

Routing Rules - Level 1

- Now we can create routing rules at the highest level.

```
5 import { Routes, Route, Navigate } from 'react-router-dom';
6 import { StudentMainPanel } from '../features/students/StudentMainPanel';
7 import { CourseMainPanel } from '../features/courses/CourseMainPanel';
8
9 function App() {
10
11 > //#region hidden old deatils...
12
13 return (
14   <div className="app-container">
15     <TopBar
16       time={currentTime}
17       onRefresh={handleRefreshTime}
18     />
19
20     <main className="main-content">
21       <Routes>
22         <Route path="/" element={<Navigate to="/students" />} />
23
24         {/* the asterisk at the end allows nested routing */}
25         <Route path="/students/*" element={<StudentMainPanel />} />
26         <Route path="/courses/*" element={<CourseMainPanel />} />
27
28         <Route path="*" element={<h1>404 - Not Found</h1>} />
29       </Routes>
30     </main>
31
32     <BottomBar />
33   </div>
34 );
35 }
```

Routing Rules - Level 1 - Explained

- The `<Routes>` element is a placeholder for interchangeable elements that depend on the URL.
- Inside, there are `<Route>` elements defining subsequent variants, as if in a grand switch/case (if/elseif/else), checked sequentially.
- The URL path is matched against the path attribute, and if the match is successful, the component from the element attribute is inserted in place of `<Routes>`.
 - `<Route path="/students/*" element={<StudentMainPanel />} />`
 - `<Route path="/courses/*" element={<CourseMainPanel />} />`
- Therefore, for **any other** address, the
 - `<Route path="*" element={<h1>404 - Not Found</h1>} />`
component is inserted, which generates an error message.
- In the case of the first line:
 - `<Route path="/" element={<Navigate to="/students" />} />`
there is a redirect from the "/" path to "/students". This is what the `<Navigate>` component is for.

Matching patterns in the path attribute

- React Router v6 uses very specific matching logic:
 - **Static Path** (`/students`): Matches only this **exact** path.
 - **Dynamic Segments** (`:id`): A **variable part** of the address. You can have **multiple** of them:
 - `path="/students/:studentId/courses/:courseId"`.
 - You can retrieve the value with the `useParams()` hook.
 - **Wildcard / Splat** (`*`): Matches **everything that follows** a given character. We use it for nested routing:
 - `path="/students/*"`.
 - We use it for 404 pages:
 - `path="*"`.
 - **Optional Segments** (`:id?`): (Reintroduced in v6.5) Allows you to define a segment that **may or may not** appear:
 - `path="/students/:id?"`.

<TopBar> Component - Code

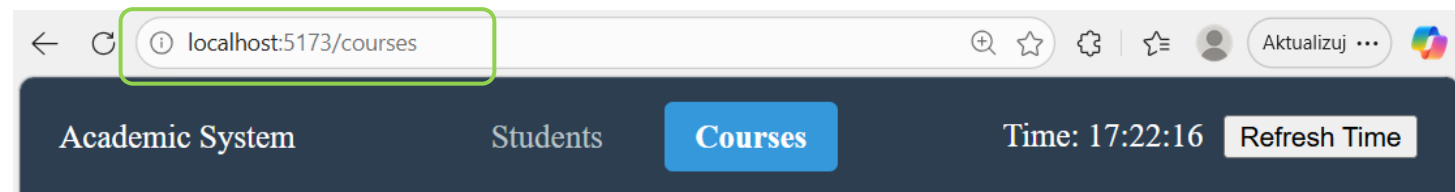
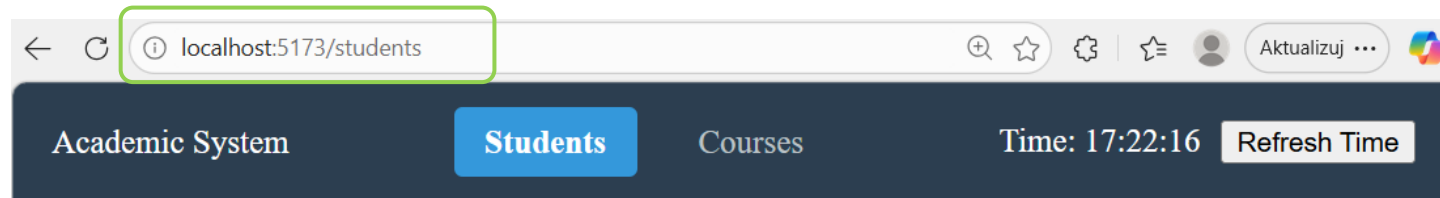
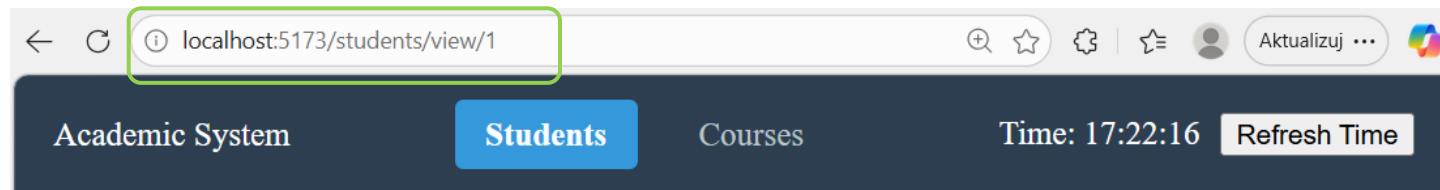
- After a little simplification, the <TopBar> should look like this :

```
2 import { NavLink } from 'react-router-dom';
3
4 > // #region hidden old deatils...
12 const TopBar = ({ time, onRefresh }: TopBarProps) => {
13   return (
14     <div>
15       <nav className="top-bar">
16         <div className="logo">Academic System</div>
17         <div className="menu">
18           <NavLink
19             to="/students"
20             className={({ isActive }) => isActive ? 'nav-link active' : 'nav-link'}
21           >
22             Students
23           </NavLink>
24
25           <NavLink
26             to="/courses"
27             className={({ isActive }) => isActive ? 'nav-link active' : 'nav-link'}
28           >
29             Courses
30           </NavLink>
31         </div>
32 > // #region hidden old deatils...
40 </nav>
41 </div>
42 );
43 };
```

```
src > components > layout > # TopBar.css > ...
1 .top-bar {
2   display: flex;
3   justify-content: space-between;
4   align-items: center;
5   padding: 0 20px;
6   background-color: #2c3e50;
7   color: white;
8   height: 60px;
9 }
10
11 .menu {
12   display: flex;
13   gap: 15px;
14 }
15
16 .nav-link {
17   color: #bdc3c7;
18   text-decoration: none;
19   padding: 8px 15px;
20   border-radius: 4px;
21   transition: 0.3s;
22 }
23
24 .nav-link:hover {
25   background-color: #34495e;
26 }
27
28 /* Class added automatically by NavLink */
29 .nav-link.active {
30   color: white;
31   background-color: #3498db;
32   font-weight: bold;
33 }
```

<TopBar> Component - Explanation

- The <NavLink> component ultimately creates an <a> element.
- Selecting/clicking a given <NavLink> navigates (local) to the page specified in the `to` attribute.
- This component also evaluates the `isActive` attribute as a **boolean** value, **indicating** that the **beginning of the current URL** matches the `to` attribute. This allows you to style the active option in the menu.



<StudentMainPanel> component - code

- using second-level routing.

```
src > features > students > StudentMainPanel.tsx > ...
1  import './StudentMainPanel.css'
2  import { Routes, Route } from 'react-router-dom';
3  import { StudentListComponent } from './StudentListComponent';
4  import { StudentViewComponent } from './StudentViewComponent';
5  import ListPlaceholder from '../common/ListPlaceholder';
6
7  export function StudentMainPanel() {
8    return (
9      <div style={{ display: 'flex', height: '80vh' }}>
10        <aside style={{ width: '30%', borderRight: '1px solid #ccc' }}>
11          <StudentListComponent />
12        </aside>
13
14        <section style={{ flex: 1, padding: '20px' }}>
15          <Routes>
16            { /* Path relative to /students */ }
17            <Route path="/" element={ <ListPlaceholder message="Select a student to view details" /> } />
18            <Route path="view/:id" element={ <StudentViewComponent /> } />
19            { /* here may be in the future: */ }
20            <Route path="create" element={ <StudentCreateComponent /> } />
21            <Route path="edit/:id" element={ <StudentEditComponent /> } />
22            { /* */ }
23          </Routes>
24        </section>
25      </div>
26    );
27  }
```

<StudentMainPanel> Component - Explanation

- The <StudentMainPanel> component has its own <Routes> element defining **relative paths**.
 - Because this element is inserted based on rules from the <App> component, the rules are executed for the "/students/*" path, so the rules apply to the part of the address containing the asterisk.
 - This time, the dynamic ":id" segment was used:
 - `<Route path="view/:id" element={<StudentViewComponent />} />`
- In the future, these rules can be extended to include additional components for creating a new student, editing, etc.
- Operations on the <StudentListComponent> element should affect the creation of the <StudentViewComponent> element, but **not** through **props** and **callbacks**, but **by changing the URL**.

<StudentListComponent> - code

src > features > students > StudentListComponent.tsx > ...

```
1  import './StudentListComponent.css'
2  import { NavLink } from 'react-router-dom';
3  import { UniversityService } from '../../services/UniversityService';
4
5  export function StudentListComponent() {
6    const students = UniversityService.getStudents();
7
8    return (
9      <div className="student-list">
10        <h3>Student list</h3>
11        <div className="list-group">
12          {students.map(s => (
13            <NavLink
14              key={s.id}
15              to={`~/students/view/${s.id}`}
16              className={({ isActive }) =>
17                isActive ? 'list-item active' : 'list-item'
18              }
19            >
20              {s.firstName} {s.lastName} (ID: {s.index})
21            </NavLink>
22          ))}
23        </div>
24      </div>
25    );
```

src > features > students > # StudentListComponent.css > ...

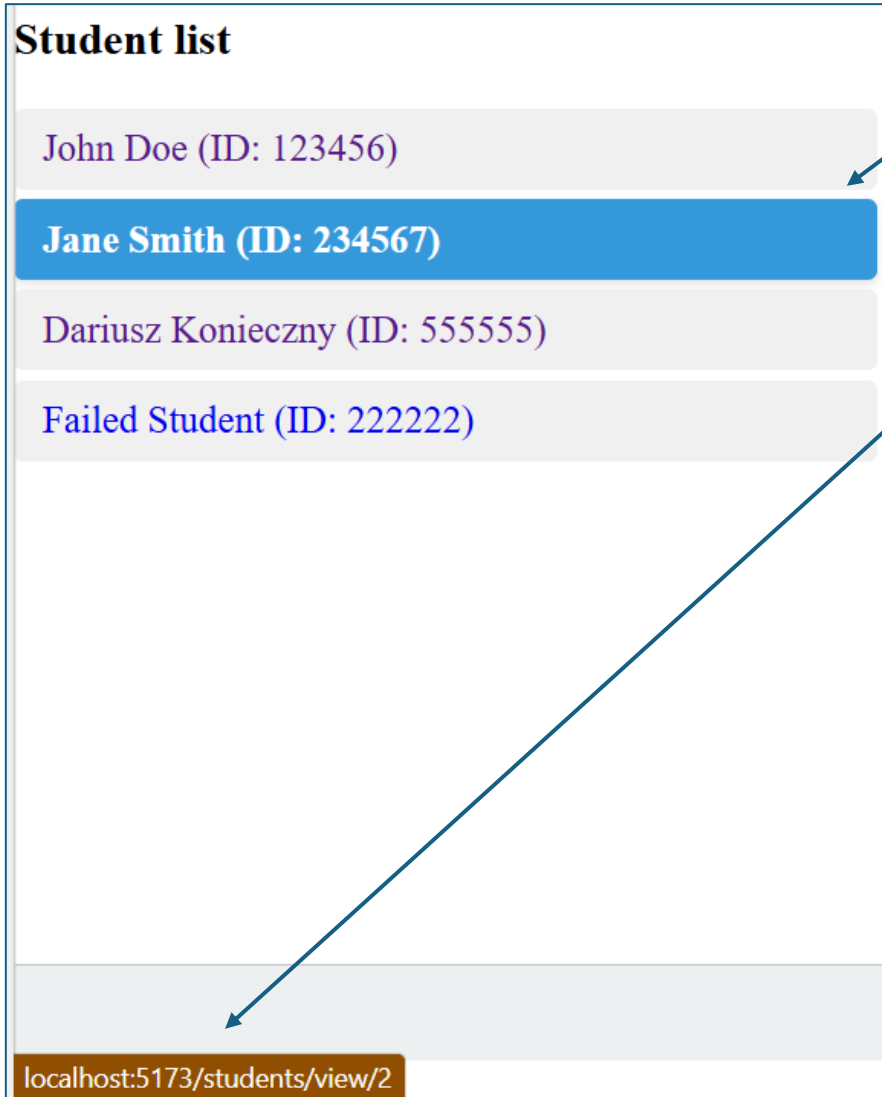
```
1  .list-group {
2    display: flex;
3    flex-direction: column;
4    gap: 10px;
5    align-items: flex-start;
6  }
7
8  .list-item {
9    text-decoration: none;
10   padding: 8px 12px;
11   border-radius: 4px;
12   background-color: #f0f0f0;
13   display: block;
14 }
15
16
17 .list-item:hover {
18   background-color: #e9ecef;
19 }
20
21 /* This is the style for the selected student */
22 .list-item.active {
23   background-color: #3498db;
24   color: white;
25   font-weight: bold;
26   box-shadow: 0 2px 4px rgba(0,0,0,0.1);
27 }
28
29 .student-id {
30   font-size: 0.8em;
31   opacity: 0.7;
32 }
```

<StudentListComponent> - explanation

- For clicking on a list item **to change the URL**, they must be `<NavLink>` components, so the `<li>` list of elements must be replaced with a `<NavLink>` "list". Therefore, there's no surrounding `<ul>` element. Styling should force these elements to appear as a **block**, as they're converted to `<a>` elements, which are typically **inline** elements.
- As before:
 - the `key` attribute to distinguish elements
 - the `to` attribute to specify the URL
 - using the calculated `isActive` attribute **to highlight** the element with an ID **equal** to the one in the **routing** as the selected element.
- So, currently, clicking on such a link generates an address like `/students/view/3`, which, according to the routing rules in the `<StudentMainPanel>` component, forces the creation of a `<StudentViewComponent>` component.

Effect

- The active element may have a different style.
- The link redirects to the appropriate URL.



<StudentViewComponent> - code changes

```
src > features > students > StudentViewComponent.tsx > ...
1  import { useParams, useNavigate } from 'react-router-dom';
2  import { UniversityService } from '../services/UniversityService';
3
4  export function StudentViewComponent() {
5    const { id } = useParams<{ id: string }>();
6    const navigate = useNavigate();
7
8    const student = UniversityService.getById(Number(id));
9
10   if (!student) return <div>Student ID: {id} not found</div>;
11
12   return (
13     <div>
14       <h2>Details: {student.firstName} {student.lastName}</h2>
15       <hr />
16       <p><strong>Department:</strong> {student.department}</p>
17       <p><strong>Icon:</strong> <img
18         src={student.icon}
19         alt={`Ikona użytkownika ${student.firstName} ${student.lastName}`}
20         style={{ width: '100px', height: '100px', borderRadius: '50%' }}
21       />
22     </p>
23     <button onClick={() => navigate('/students')}
24       style={{ marginTop: '20px', color: 'red' }}>
25       Close preview
26     </button>
27   </div>
28   );
29 }
```

<StudentViewComponent> - explanation

- Since the student ID information is in the routing rule, the target component can read it using the `useParams()` hook, which returns an object with `key-value pairs` derived **from** parsing the **URL path**.
- The code uses deconstruction into variable members (here: `id`).
- This also causes the component to read data for the student with the given ID from the website and use it to create the JSX.
- The `useNavigate()` method returns a function to force a URL change, similar to the `<Navigate>` component.
- Both of these methods are **library hooks** of the `react-router-dom` module.
 - Internally (in their code), they use **built-in hooks**.

Other components - placeholders

```
src > features > courses > CourseMainPanel.tsx > ...
1 import CourseListComponent from './CourseListComponent'
2 import ListPlaceholder from '../common/ListPlaceholder'
3 import { Route, Routes } from 'react-router-dom';
4
5 export function CourseMainPanel() {
6   return (
7     <div style={{ display: 'flex', height: '80vh' }}>
8       <aside style={{ width: '30%', borderRight: '1px solid #ccc' }}>
9         <CourseListComponent />
10      </aside>
11
12      <section style={{ flex: 1, padding: '20px' }}>
13        <Routes>
14          <Route path="/" element={<ListPlaceholder message="Course preview coming soon!" />} />
15        </Routes>
16      </section>
17    </div>
18  );
19 }
```

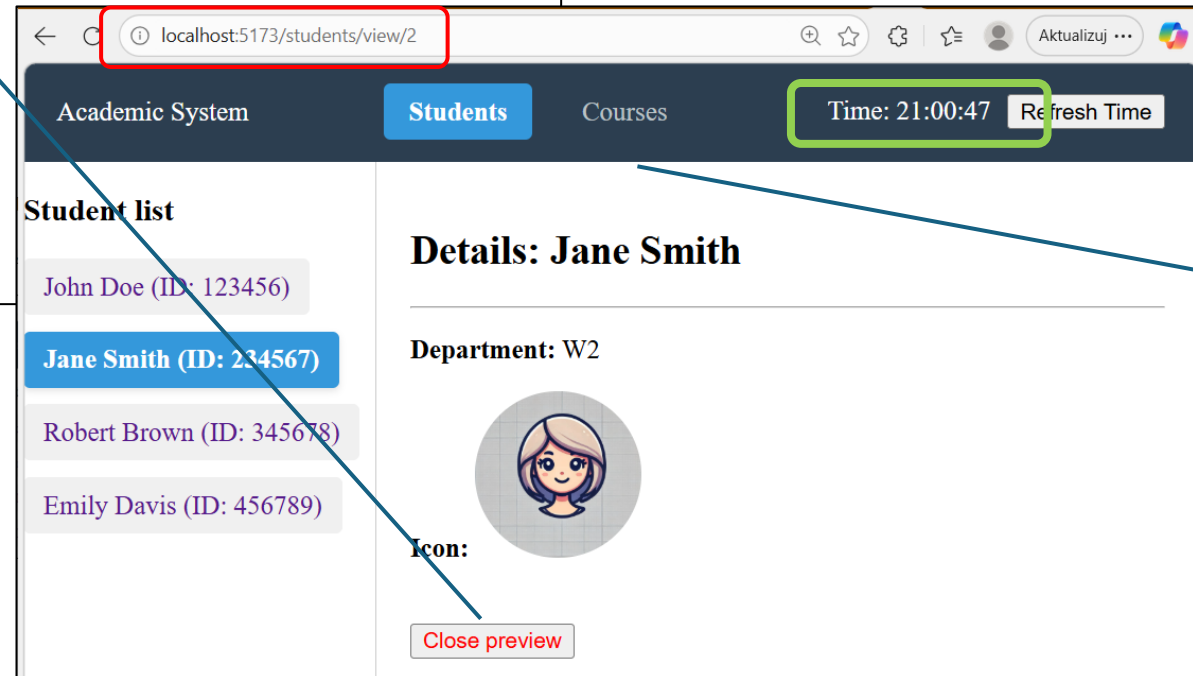
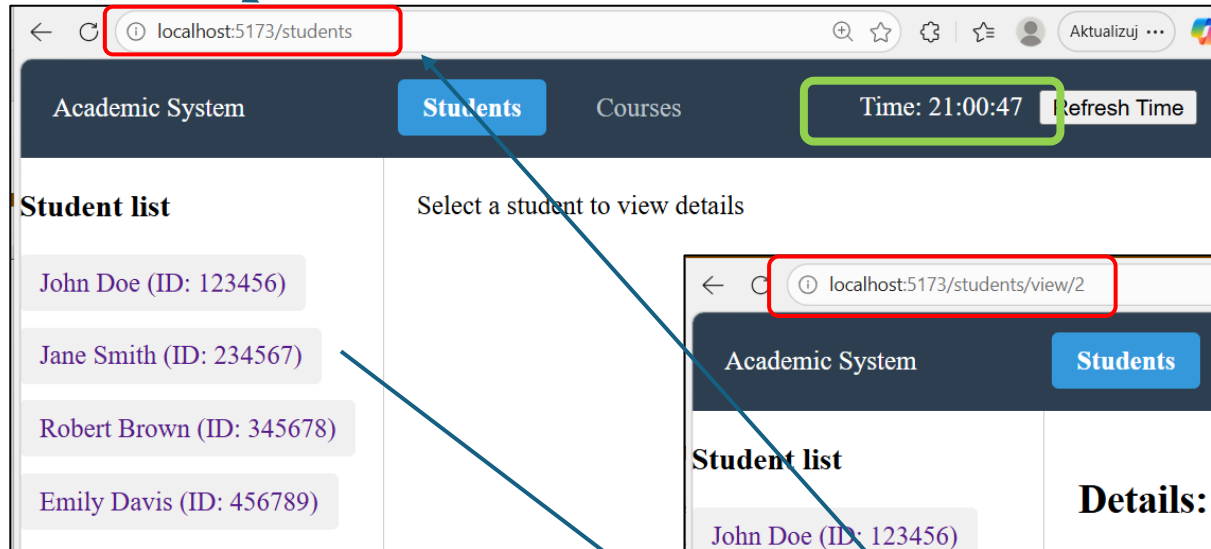
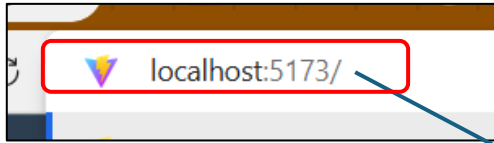
```
src > features > courses > CourseListComponent.tsx > ...
1 const CourseListComponent = () => {
2   return (
3     <div>
4       <CourseListComponent placeholder />
5     </div>
6   )
7 }
8
9 export default CourseListComponent
```

```
export const UniversityService = {
  getStudents: (): Student[] => {
    return [...MOCK_STUDENTS];
  },
  getStudentById: (id: number): Student | undefined => {
    return MOCK_STUDENTS.find(s => s.id === id);
  }
};
```

```
src > common > ListPlaceholder.tsx > ...
1 //import React from 'react'
2
3 interface ListPlaceholderProps {
4   message : string;
5 }
6
7 const ListPlaceholder = ({message} : ListPlaceholderProps) => {
8   return (
9     <div>{message}</div>
10  )
11 }
12
13 export default ListPlaceholder
```

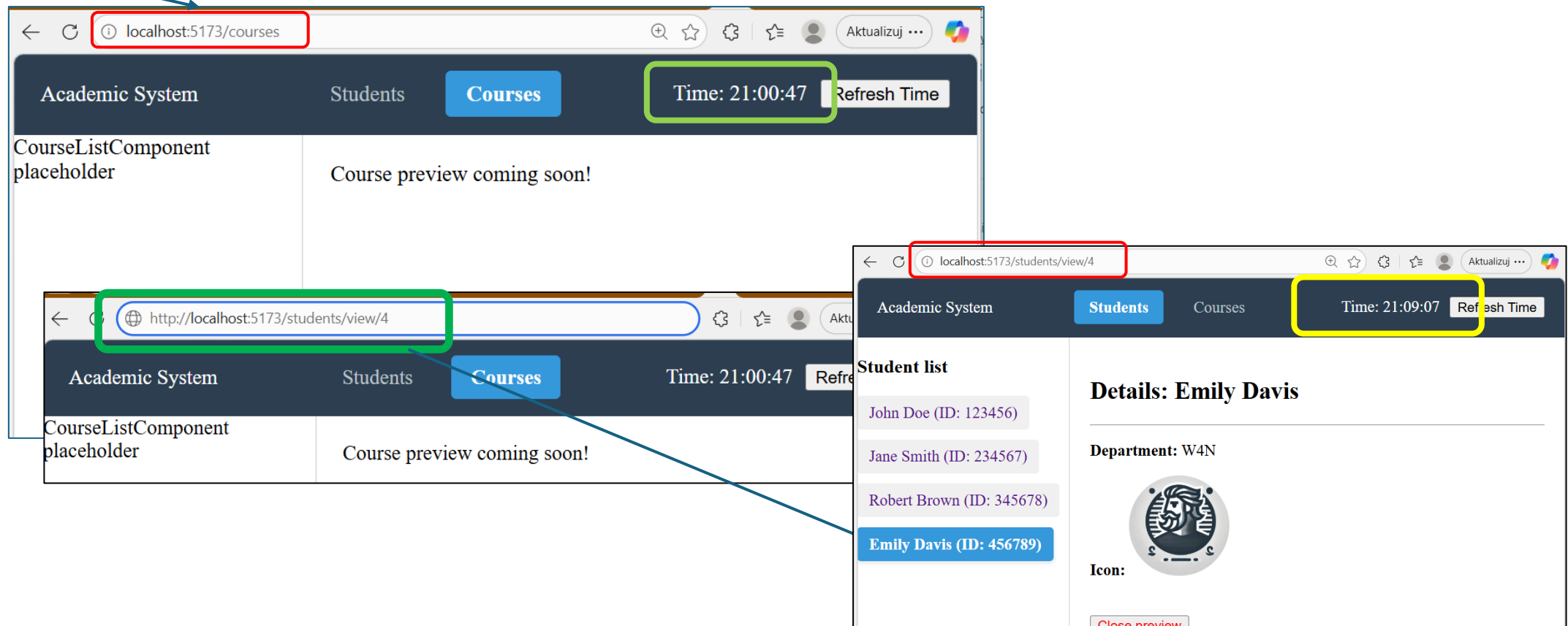
- <CourseMainPanel> component with a pre-defined <Routes> component for routing related to the course list (similar to the student collection).
- <CourseListComponent> component with direct placeholder code.
- <ListPlaceholder> component with a message prop for any empty lists.
- Adding a new **getStudentById()** method to the service.

Obtained action 1/2



Obtained action 2/2

- Everything works as intended. Furthermore, observing the time shows that changing the URL using the buttons in the app doesn't refresh the entire page. On the other hand, we can enter the URL at any time, which not only downloads and launches the entire application but also redirects us to the appropriate components based on the address.



Forms

Example: `<addStudentComponent>`

Using local hooks

Adding a new student

- To permanently manage changes, use states (`useState()` method) **for each editable** student field.
- Each change to a field's value in the form **should update** the corresponding state.
- Selecting the **faculty** and **icon** should be done using a **drop-down list**, as this will be the most convenient.
- The actual adding of a student should be performed **by the service** responsible for managing the student collection.
- The form should open in the **right** panel at `/students/add`.
- In the graphical interface, there should be a "+ Add student" **button** above the student list.

<AddStudentForm> component - code 1/3

- We need to use states again, this time for **internal (local)** behavior.
- **Each** form field has a separate state.
- We need to prepare a **function** that will be called when the form is "**submitted**," which will receive a `React.FormEvent` as a parameter.
 - First, we need to **block the default** form submission **behavior**. Without this, the browser would refresh the page after clicking "Save Student," which would clear the application state (all students in memory).
 - Then we need to **repackage** the data from the states into an object of the `Student` class.
 - **Save** this object using a **service method**.
 - **Redirect** the application to the `"/students"` page.

```
src > features > students > AddStudentForm.tsx > ...
1  import React, { useState } from "react";
2  import { useNavigate } from "react-router-dom";
3  import { type Student } from "../../models/Student";
4  import { type Department } from "../../models/Department";
5  import { UniversityService } from "../../services/UniversityService";
6  import './AddStudentForm.css';
7
8  export function AddStudentForm() {
9    const navigate = useNavigate();
10
11    // Status for the form fields
12    const [firstName, setFirstName] = useState("");
13    const [lastName, setLastName] = useState("");
14    const [indexNumber, setIndexNumber] = useState<number>(0);
15    const [department, setDepartment] = useState<Department>("W1");
16    const [icon, setIcon] = useState("/icons/icon1.png");
17
18    const handleSubmit = (e: React.FormEvent) => {
19      e.preventDefault();
20
21      const newStudent: Student = {
22        id: Date.now(), // Simple ID generation for demo purposes
23        firstName,
24        lastName,
25        index: indexNumber,
26        department,
27        icon,
28      };
29
30      UniversityService.addStudent(newStudent);
31      // Return to list after adding
32      navigate("/students");
33    };
34  }
```

<AddStudentForm> component - code 2/3

- In the form:
 - We **connect** the prepared method to the form's submit event.
 - Whenever the `firstName`, `lastName`, and `indexNumber` fields change (`onChange` attribute) from the form, we update the state value of the corresponding state, optionally converting it to the appropriate type for the given state:
`Number(e.target.value)`

```
35   return (  
36     <div className="add-student-form">  
37       <h2>Add a new student</h2>  
38       <form onSubmit={handleSubmit}>  
39         <div className="form-group">  
40           <label>First name:</label>  
41           <input  
42             type="text"  
43             value={firstName}  
44             onChange={(e) => setFirstName(e.target.value)}  
45             required  
46           />  
47         </div>  
48  
49         <div className="form-group">  
50           <label>Last name:</label>  
51           <input  
52             type="text"  
53             value={lastName}  
54             onChange={(e) => setLastName(e.target.value)}  
55             required  
56           />  
57         </div>  
58  
59         <div className="form-group">  
60           <label>Index No:</label>  
61           <input  
62             type="number"  
63             value={indexNumber}  
64             onChange={(e) => setIndexNumber(Number(e.target.value))}  
65             required  
66           />  
67         </div>  
    )
```

<AddStudentForm> component - code 3/3

- The next fields are drop-down lists. Because there are a small, fixed number of options, they can be created manually.
 - Otherwise, it would be worth using a mapping from a collection of objects to a list of `<option>` elements.
 - Because the `Department` type is a union of strings, which is not the same as any string, using the `as` operator yields the desired data type.
 - For the icon, the `<img>` element is used to read the icon state, which, thanks to the state mechanism, is updated whenever the icon selection in the list changes.
- For the "cancel" button, another use of the `navigate()` method is presented
 - with a **negative numeric** parameter indicates how many entries in the **page visit history** to go back.

```
69     <div className="form-group">
70       <label>Department:</label>
71       <select
72         value={department}
73         onChange={(e) => setDepartment(e.target.value as Department)}
74       >
75         <option value="W1">Department W1</option>
76         <option value="W2">Department W2</option>
77         <option value="W3">Department W3</option>
78         <option value="W4N">Department W4N</option>
79       </select>
80     </div>
81
82     <div className="form-group">
83       <label>Profile icon:</label>
84       <select value={icon} onChange={(e) => setIcon(e.target.value)}>
85         <option value="/icons/icon1.png">Icon 1</option>
86         <option value="/icons/icon2.png">Icon 2</option>
87         <option value="/icons/icon3.png">Icon 3</option>
88         <option value="/icons/icon4.png">Icon 4</option>
89       </select>
90       <img
91         src={icon}
92         alt="Preview of selected icon"
93         style={{ width: "40px", marginLeft: "10px" }}
94       />
95     </div>
96
97     <div className="form-actions">
98       <button type="button" className="btn-cancel" onClick={() => navigate(-1)}>
99         Cancel
100       </button>
101       <button type="submit" className="btn-save">
102         Save student
103       </button>
104     </div>
105   </form>
106 </div>
107 );
108 }
```

Modifying <StudentMainPanel>

- Adding a "/add" relative link to the <AddStudentForm> component.
- You can also replace the path="/" with the index attribute.

```
6 import { AddStudentForm } from "../AddStudentForm";
7
8 export function StudentMainPanel() {
9   return (
10     <div style={{ display: 'flex', height: '80vh' }}>
11       <aside style={{ width: '30%', borderRight: '1px solid #ccc' }}>
12         <StudentListComponent />
13       </aside>
14
15       <section style={{ flex: 1, padding: '20px' }}>
16         <Routes>
17           {/* Path relative to /students */}
18           <Route index element={<ListPlaceholder message="Select a student to view details" />} />
19           <Route path="view/:id" element={<StudentViewComponent />} />
20           <Route path="add" element={<AddStudentForm />} />
21         </Routes>
22       </section>
23     </div>
24   );
25 }
```

Modifying <StudentListComponent>

- adding a "+ Add Student" button redirecting the application to the "/students/add" page.

```
5 export function StudentListComponent() {  
6   const students = UniversityService.getStudents();  
7   const navigate = useNavigate();  
8  
9   return (  
10    <div className="student-list">  
11      <button className="add-student"  
12        onClick={() => navigate('/students/add')}>  
13        + Add student  
14      </button>  
15      <h3>Student list</h3>  
16      <div className="list-group">  
17        {students.map(s => /
```

Effect 1/2

- In the meantime, you can refresh the time.
 - The form loses focus, but not the entered data.

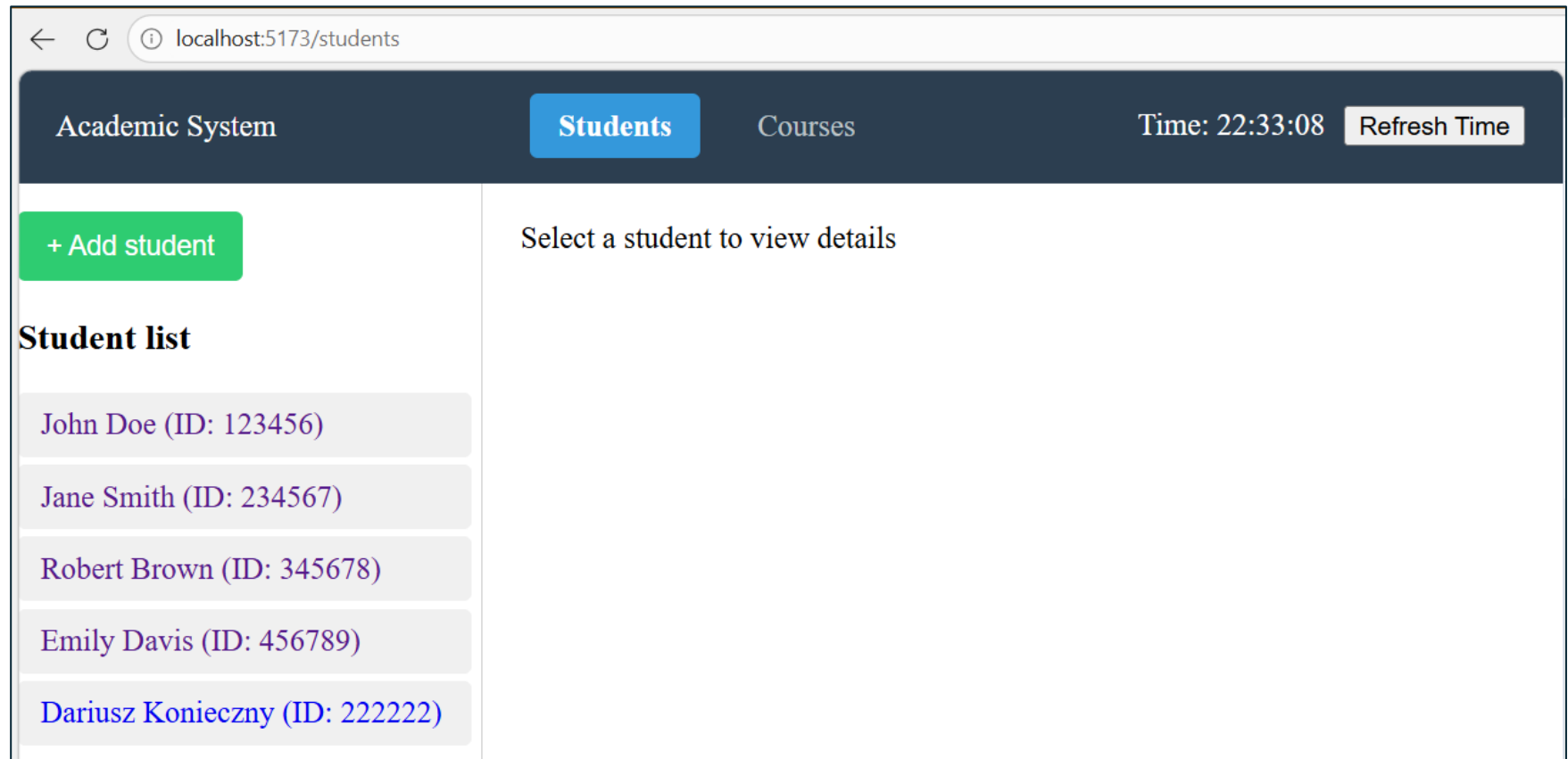
The image displays two screenshots of a web application interface, illustrating the state of a form during a refresh operation.

Left Screenshot: The application is titled "Academic System" and shows a "Students" tab. The "Add a new student" form is visible, with fields for First name, Last name, Index No, Department, and Profile icon. The form is currently empty. A red arrow points to the "Refresh Time" button in the top right corner.

Right Screenshot: The same application is shown, but the "Add a new student" form now contains data: First name: Dariusz, Last name: Konieczny, Index No: 222222, Department: Department W4N, and Profile icon: Icon 3. A red arrow points to the "Refresh Time" button in the top right corner.

Effect 2/2

- After clicking the "Save student" button:
 - you will be redirected to the appropriate page.
 - The new student will appear on the list.



Asynchronous, communication with the backend

`fetch()` method

`async/await` operators, `Promise<T>`

Hooks: `useLocation()`, `useEffect()`

`json-server` tool

External business data - asynchronous

- The current version of the program only remembers new data while the program is running on a specific user's computer.
- If such data truly needs to be stored **separately** for **each** user, `localStorage` should be used.
- A much more important issue is the use of a **shared** database.
- SPA applications don't directly access a database, but rather a backend application that provides them with some API, typically an API controller using a RESTful approach.
- This also means that receiving such data can take a **noticeably long time**, so **asynchronous** programming should be used,
 - both when implementing the **service** responsible for communicating with the backend,
 - and in **component** functions, as different components can use different services simultaneously, and one component shouldn't wait for another while rendering to the virtual DOM.
 - We should start using so-called **spinners**, i.e., JSX, which are created while the user is waiting for a response from the backend, and JSX with an error message if communication fails.

<UniversityService> - asynchronous version

- Asynchronous connections in React, of course, use AJAX. However, not explicitly, but much more easily through the asynchronous method `responseObject=fetch(url, requestObject)`.
- If the **second** argument is **missing**, it is assumed that there are **no headers**, the message **body** is **empty**, and the request is sent via the **GET** method.
- **Otherwise** in this object, you should fill in the fields **named method, headers, and body** as needed.

```
d > src > services > TS UniversityService.ts > ...
import type { Student } from "../models/Student";

const API_URL = "http://localhost:5000/students";

export const UniversityService = {
  // GET /students
  getStudents: async (): Promise<Student[]> => {
    const response = await fetch(API_URL);
    if (!response.ok) throw new Error("Error downloading student list");
    return response.json();
  },

  // GET /students/:id
  getStudentById: async (id: number): Promise<Student | undefined> => {
    const response = await fetch(`${API_URL}/${id}`);
    if (response.status === 404) return undefined;
    if (!response.ok) throw new Error("Error downloading student data");
    return response.json();
  },

  // POST /students
  addStudent: async (student: Student): Promise<Student> => {
    const response = await fetch(API_URL, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(student),
    });
    if (!response.ok) throw new Error("Error saving student data");
    return response.json();
  }
};
```

addStudent () method

```
// POST /students
addStudent: async (student: Student): Promise<Student> => {
  const response = await fetch(API_URL, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(student),
  });
  if (!response.ok) throw new Error("Error saving student data");
  return response.json();
}
```

- In the case of `addStudent ()` , the second argument specifies the most important elements:
 - `method`: this time "POST"
 - `headers`: an object containing header names and their values, here only one pair: the "Content-Type" header with the value "application/json"
 - `body`: the body of the request. Here, the student object converted to JSON format.

A Simplified Explanation of Concurrency

- The asynchronous operator `await` means stopping the current thread and placing it on a microtask queue until the `Promise` resolves.
 - In this case, until the data is read from the backend server. However, the request itself is executed by the browser engine (not JavaScript), so in the meantime, the browser can submit another microtask to the JavaScript engine, for which the `Promise` has been resolved in the meantime.
 - This task then resumes execution from where it was stopped.
- The `await` operator can only be used before a method called with the keyword `async`. Furthermore, this means that although such a method returns a `Promise` of type `X` (`Promise<X>`), a value of type `X` is returned at the point of such a call.
- An asynchronous method can be called **without** the keyword `await`. However, this means that a value of type `Promise<X>` will be returned **almost immediately**, and the thread will **not be paused**. Such a promise is then in the **Pending** state because it has not yet been resolved.
 - This allows for your own, more flexible use of multitasking, which is not part of this lecture.
 - This approach will not be used.

Resolution of the promise

- Resolving a promise using `await` can end in two ways.
 - Success – a **value** of type `X` will be returned.
 - Error – an **exception** will be thrown.
- So, if an exception is likely to occur, the code calling `await` should be in a `try...catch...` context.
 - Unless we don't want to handle the exception at this level, as in the code presented. However, this ends with the user receiving a strange message: "Failed to fetch." Sometimes it's worth **catching** such an exception and **changing the message**, for example, to "Connection problem. Check if the server is running."

Result of the `fetch()` method

- The result of the `fetch()` method is an object of the `Promise<Response>` class.
- The `Response` class has getters/fields/methods:
 - `ok` - returns true if the HTTP status is in the range 200-299
 - `status` - HTTP status code (e.g., 200, 404, 500)
 - `headers` - Response headers sent by the server.
 - `json()` - an asynchronous method returning the `Promise` with the result from the body as a JS object (also an array)
 - `text()` - an asynchronous method returning the `Promise` with the result from the body as a string
- Why are the **last two** methods **asynchronous**? Because `fetch()` terminates as soon as it **reads** the response **header**.
- Therefore, the last two methods read the data stream contained in the body, which can take a long time.
 - If communication encounters **a problem** while reading the body, these methods will throw an **exception**.

Calling service methods in components

- All components that previously retrieved data from the service synchronously must now do so asynchronously. The new solution **must anticipate** the **various states** caused by this change:
 - **Data not yet read** - set a **placeholder** with information for the user
 - **Data read successfully** - populate the **component with data** as for the synchronous version
 - **Communication with the backend failed** - set a **placeholder** with this information
- Use a mechanism **to detect changes** in the component's state.
 - The `useState()` method will be used to internally store states, as well as new hooks: the `useLocation()` and `useEffect()` methods.

<StudentListComponent> - code changes

- students/setStudents state - to store the student array.
- loading/setLoading state - to remember whether we are in the process of loading data from the service/backend.
- error/setError - to remember whether the loading operation ended with an error. This state is significant if loading==**false**.
- Lines 27 to the end describe the component creation depending on the situation defined by these two states. If everything is OK, i.e., the last return, the code has not changed.

```
2 import { NavLink, useLocation, useNavigate } from 'react-router-dom';
3 import { UniversityService } from '../services/UniversityService';
4 import { useEffect, useState } from "react";
5 import type { Student } from "../models/Student";
6
7 export function StudentListComponent() {
8   const [students, setStudents] = useState<Student[]>([]);
9   const [loading, setLoading] = useState(true);
10  const [error, setError] = useState<string | null>(null);
11  const navigate = useNavigate();
12
13  const location = useLocation(); // 2. We get the location object
14
15  useEffect(() => {
16    UniversityService.getStudents()
17      .then(data => {
18        setStudents(data);
19        setLoading(false);
20      })
21      .catch(err => {
22        setError(err.message);
23        setLoading(false);
24      });
25  }, [location]);
26
27  if (loading) return <div>Loading data...</div>;
28  if (error) return <div className="error">{error}</div>;
29
30  return (
```

Hook `useLocation()`

- Only the **getter** has a `location` hook, as it's changed by
 - changing the page address in the browser
 - invoking a change in address via the `navigate()` or `<navLink>` methods
- The **change** occurs even if the new address is the same as the previous one (e.g., via **reload**)!
 - This is forced by changing the `key` field in the returned object.
- The getter returns a native object with the following fields:

```
{
  pathname: "/lecture", // e.a. "/user/profile"
  search: "?id=10",     // querystring parameters
  hash: "#bio",         // anchor after # symbol
  state: { ... },       // Your own data sent via navigate()
  key: "ah367j"         // UNIQUE ID for this particular entry
}
```

Hook `useEffect()`

- Reminder: The `useState()` hook used in a component function causes **its change to re-run** the component's method.
- The `useEffect()` hook has two call parameters:
 - a **method** with no arguments to execute
 - an **array** of hooks (or no parameter)
- Running the **method does not re-run** the component's creation method.
- If the hook array **contains any hooks**, changing **any of them** causes the `useEffect()` method to be called (in the appropriate phase).
- If the array is **empty**, the method will be called **once** when the component **becomes visible** to the user.
- If the array is **missing** (`undefined`), the method will be executed **each time** the object **becomes visible** after the next re-rendering.
 - In this case, the method **should not change** the component's creation method, as this will **force a call** to the component's **creation method**, which will **force** another call to `useEffect()`, and so on.

Using `useEffect()` - code

```
useEffect(() => {  
  UniversityService.getStudents()  
    .then(data => {  
      setStudents(data);  
      setLoading(false);  
    })  
    .catch(err => {  
      setError(err.message);  
      setLoading(false);  
    });  
}, [location]);
```

- For the `<StudentListComponent>` component code, the `useEffect()` method from the hook responds to any `location` change.
- The method reads student data from the service using the asynchronous `getStudents()` method. This time, we use the Fluent API, using methods from the `Promise<T>` class:
 - `.then(data=> { code if success })`
 - `.catch(err=> { code if exception })`
- The `.then()` method will "wait" for the promise to be fulfilled before it starts executing.
- Only after these codes trigger the appropriate setters for flag or student collection state changes, which only trigger the component creation method to be called again.

The actual operation of the `await` operator

```
useEffect(() => {  
  UniversityService.getStudents()  
    .then(data => {  
      setStudents(data);  
      setLoading(false);  
    })  
    .catch(err => {  
      setError(err.message);  
      setLoading(false);  
    });  
}, [location]);
```

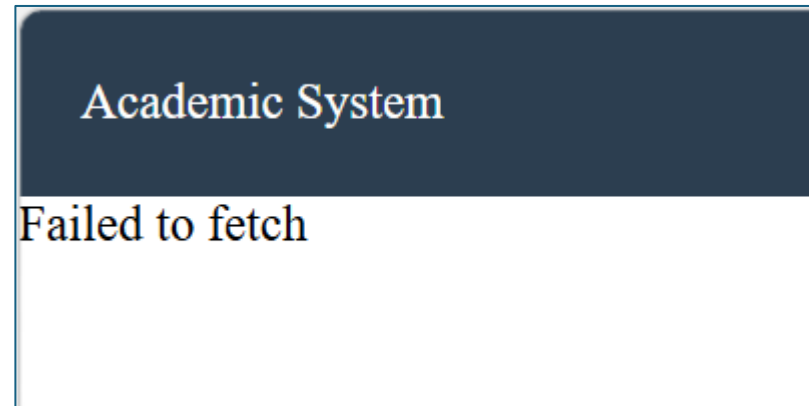
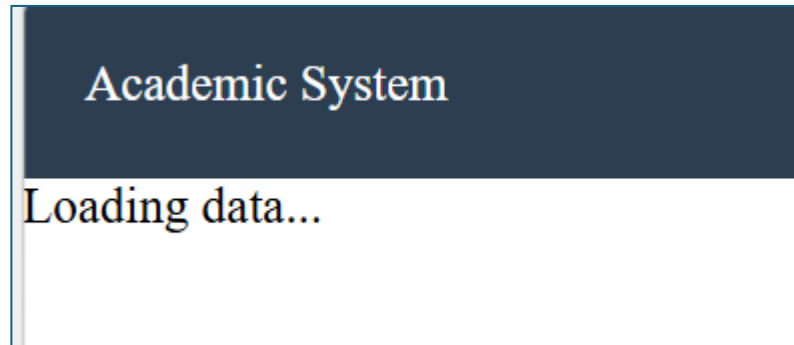
```
// GET /students  
getStudents: async (): Promise<Student[]> => {  
  const response = await fetch(API_URL);  
  if (!response.ok) throw new Error("Error downloading student list");  
  return response.json();  
},
```

Browser engine
(true multi-threaded processing)

- JavaScript processing is truly single-threaded.
- The `await` operator doesn't immediately halt the execution of an asynchronous method; it continues executing internal code until it finally reaches an **asynchronous method**, which is actually **executed by the browser** (in this case: `fetch()`). **Only then** does it **suspend** the current JavaScript code and execute some other code that isn't suspended. At the same time, actual **concurrent processing** occurs, but **through the browser** (in this case: communication with the backend). When communication completes (successfully or with an exception), that thread is placed on a **queue of threads** to be **resumed**. And when it leaves the queue, code execution resumes.

The effect of the program

- The left screenshot shows a message related to `loading==true`.
- The right screenshot shows that after a while, `loading==false`, and `error=="Failed to fetch"`
 - We don't have the backend running.



Temporary backend - `json-server`

- The backend can be written in Spring Boot, Node.js/Express.js, and many other technologies – this was covered in previous lectures.
- However, you can quickly launch a `json-server` that can simulate the operation of a RESTful server.
- To run, it requires a **single JSON file** with data (the "database"), which it parses and generates the appropriate endpoints for the GET/POST/PUT/DELETE methods.
- Operations that change the state of the "database" will change the file's content(for `--watch` option).
- Based on the field names, it "guesses" which field is the primary key, but it also guesses foreign keys.
 - During POST operations, it generates random primary keys.
- By launching such a server, you can set a **response delay** (the `--delay` option), making it easier to test the application's behavior when it **waits longer** for a response.
- It is launched using NPX (Node Package eXecute).
- The `json-server` tool can do much, much more.

Files to execute

```
back-end
  {} db.json
  {} run.bat
```

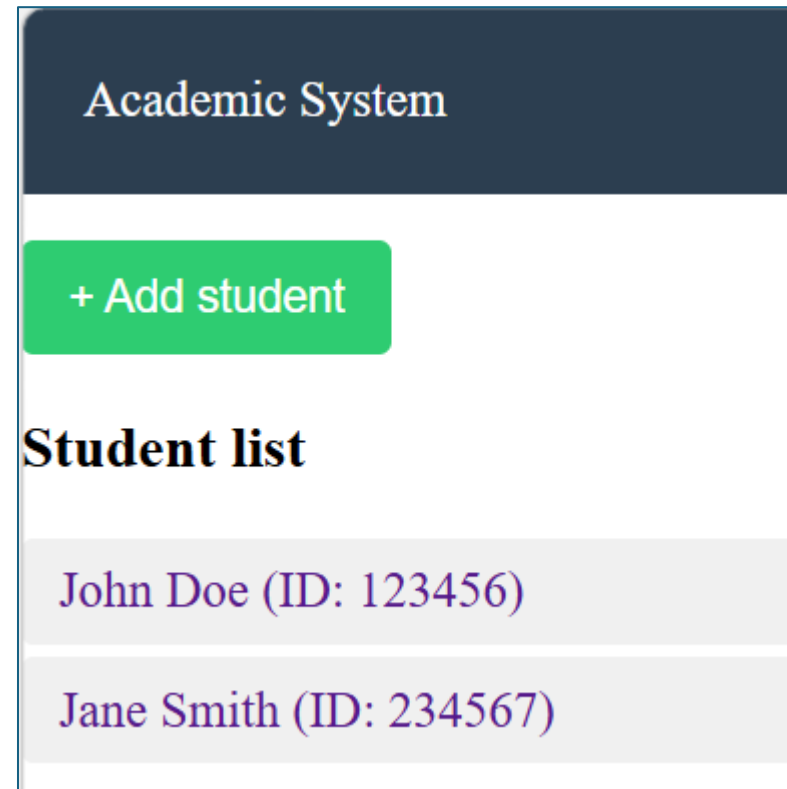
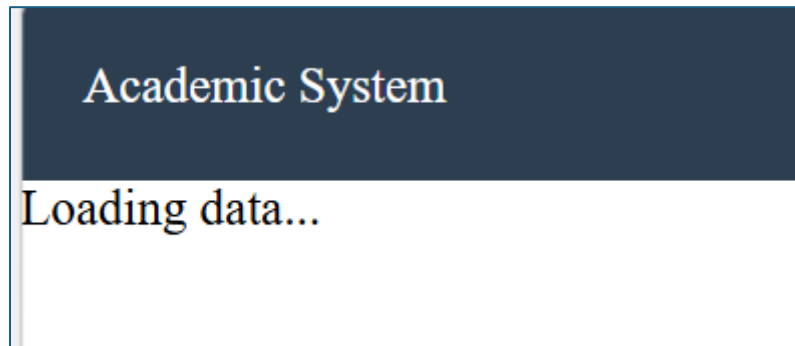
```
back-end > run.bat
1 npx json-server@0.17.4 --delay 2000 --port 5000 db.json --watch
```

```
back-end > {} db.json > ...
1 {
2   "students": [
3     {
4       "id": "1",
5       "index": 123456,
6       "firstName": "John",
7       "lastName": "Doe",
8       "department": "W1",
9       "icon": "/icons/icon1.png"
10    },
11    {
12      "id": "2",
13      "index": 234567,
14      "firstName": "Jane",
15      "lastName": "Smith",
16      "department": "W2",
17      "icon": "/icons/icon2.png"
18    }
19  ]
20 }
```

- The server must be installed first.
 - It's best to run it in a **folder other** than the front-end application.
- The `--delay` option allows you to specify a delay for each response in **milliseconds** (i.e., above 2 seconds).
- The `--watch` option means that **changes** should be **written** to the file **immediately**.
- The following endpoints are created:
 - GET `/api/students`
 - GET `/api/students/:id`
 - POST `/api/students`
 - PUT `/api/students/:id`
 - DELETE `/api/students/:id`

Application operation after backend launch

- The left screenshot shows the message related to `loading==true`.
- The right screenshot shows that after a while (2s), `loading==false`, `error==""` and `useEffect().then()` will work.



<StudentViewComponent> - code changes

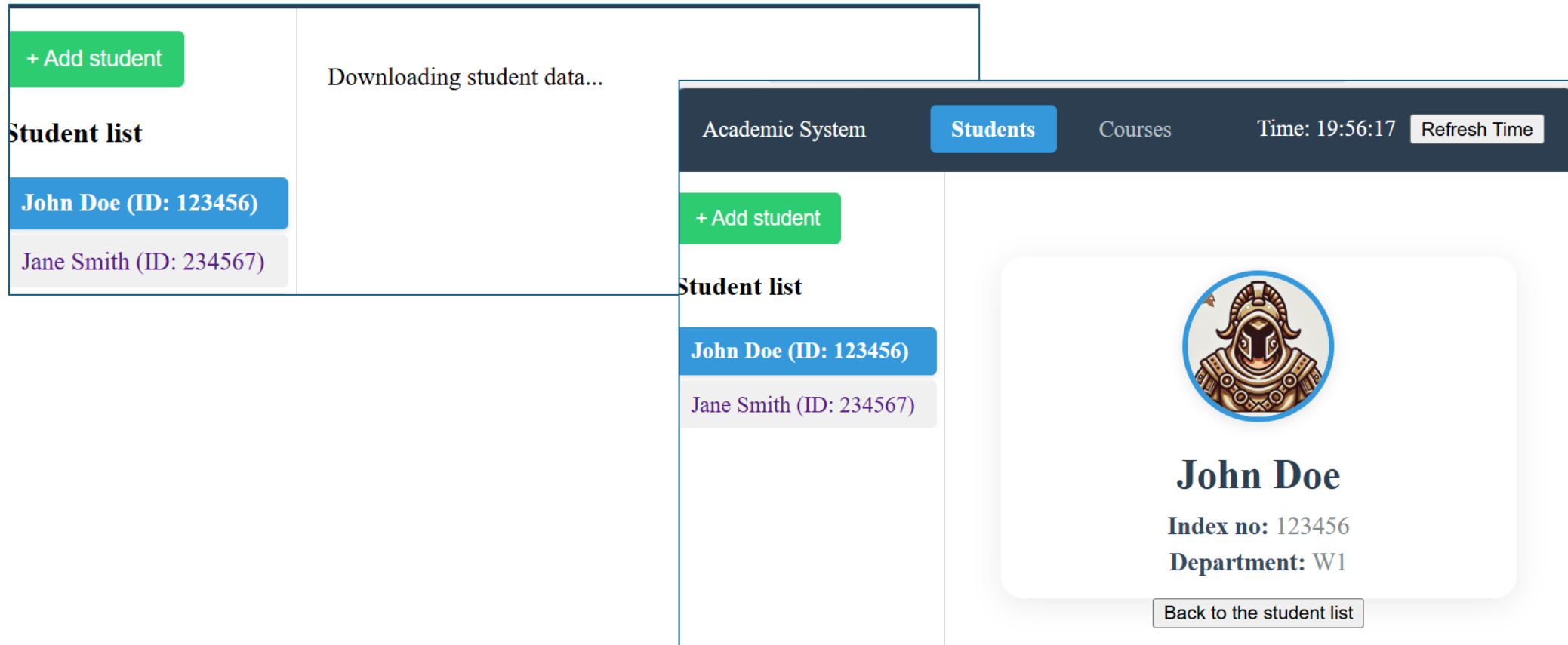
```
export function StudentViewComponent() {  
  const { id } = useParams<{ id: string }>();  
  const navigate = useNavigate();  
  
  const [student, setStudent] = useState<Student | null>(null);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState(false);  
  
  const handleClose = () => {  
    navigate('/students');  
  };  
  
  useEffect(() => {  
    const fetchStudent = async () => {  
      setLoading(true);  
      setError(false);  
      try {  
        const studentId = Number(id);  
        const data = await UniversityService.getStudentById(studentId);  
  
        if (data) {  
          setStudent(data);  
        } else {  
          setError(true);  
        }  
      } catch {  
        setError(true);  
      } finally {  
        setLoading(false);  
      }  
    };  
  
    fetchStudent();  
  }, [id]); // Re-download when the ID in the URL changes
```

- Similar changes should be made to the <StudentViewComponent> component using the `getStudentById()` method, except that in this case, you simply need to observe the changes in the hook `id` in `useEffect()`.
- The code for the last `return` has not changed.

```
if (loading) {  
  return (  
    <div className="spinner-container">  
      <div className="loading-spinner"></div>  
      <p>Downloading student data...</p>  
    </div>  
  );  
}  
  
if (error || !student) {  
  return (  
    <div className="error-container">  
      <p>No student with the given ID was found.</p>  
      <button onClick={() => navigate('/students')}>Back to list</button>  
    </div>  
  );  
}  
  
return
```

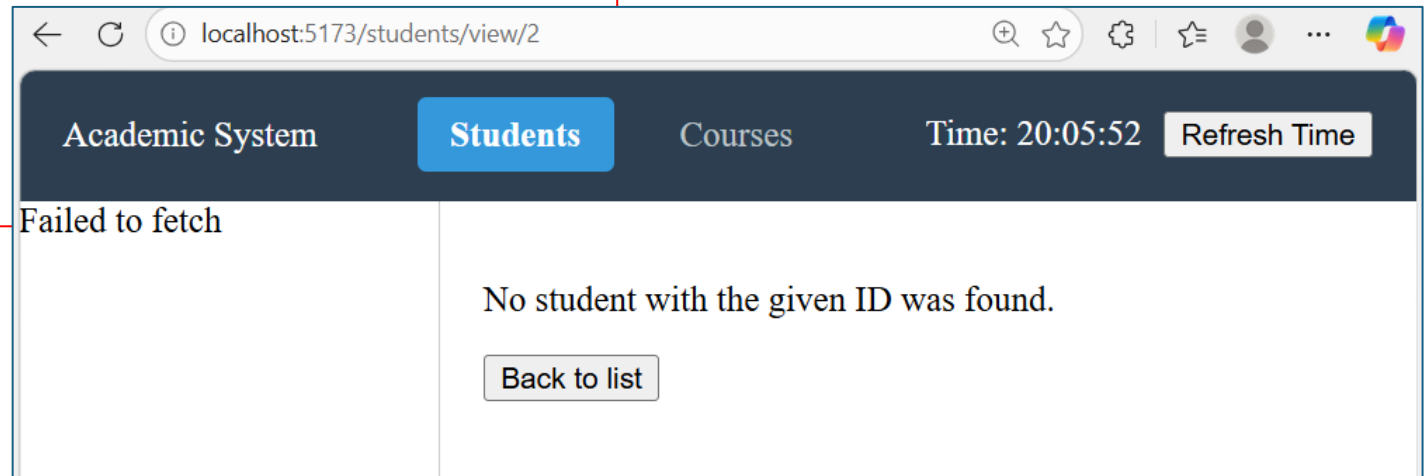
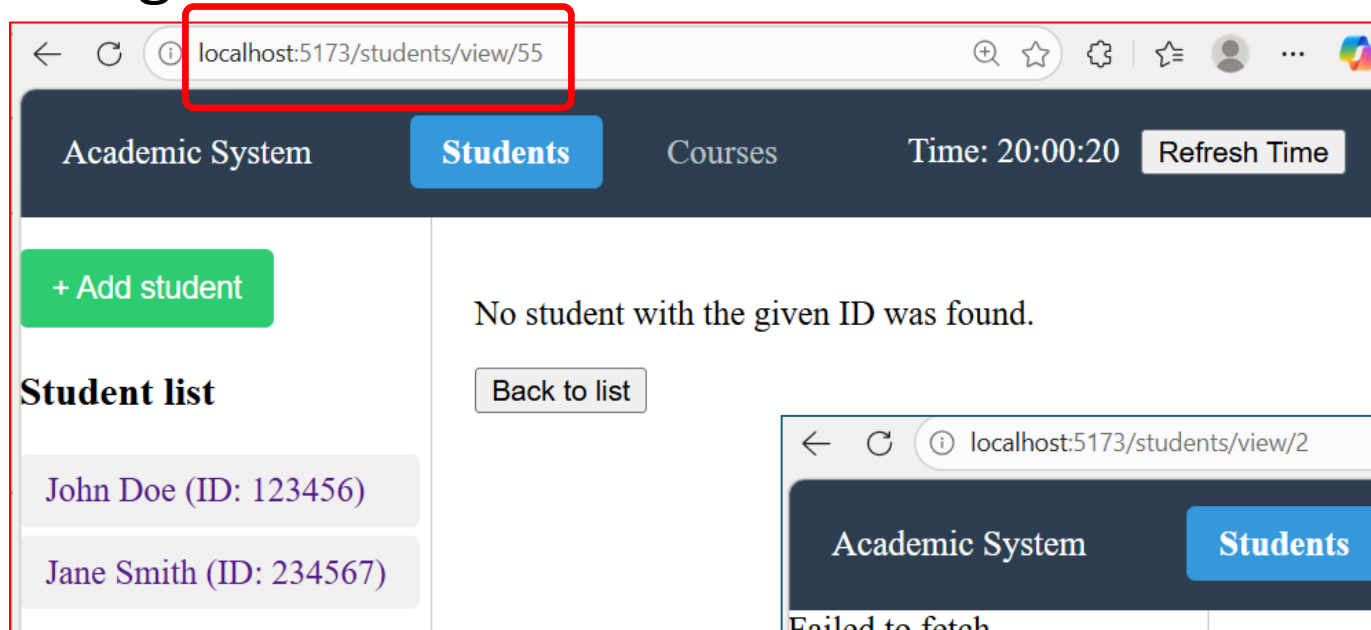
The effect of the program 1/2

- The left screenshot shows the message related to `loading==true`.
- The right screenshot shows that after a while, `loading==false`, `error==""` and `useEffect()` will be triggered, including the `getStudentById()` method.



The effect of the program 2/2

- Left screenshot - nonexistent ID in the URL.
- Right screenshot - backend disabled.



Changes to <AddStudentForm> 1/2

```
export function AddStudentForm() {  
  // #region old code...  
  // Additional state to lock the button while sending  
  const [isSubmitting, setIsSubmitting] = useState(false);  
  
  const handleSubmit = async (e: React.FormEvent) => {  
    e.preventDefault();  
    setIsSubmitting(true);  
  
    const newStudent: Student = {  
      id: Date.now(), // json-server will assign the ID it  
      firstName,  
      lastName,  
      index: indexNumber,  
      department,  
      icon  
    };  
  
    try {  
      await UniversityService.addStudent(newStudent);  
      // After success we return to the list  
      navigate('/students');  
    } catch (error) {  
      console.error("Error details:", error);  
      alert("An error occurred while adding a student!");  
      setIsSubmitting(false);  
    }  
  };  
}
```

- When adding a new student, you need to consider several situations that may occur when attempting to save data to the service.
 - Saving is in progress
 - **Placeholder** for this situation, **spinner**
 - During the saving process, the user can click "Save student" or "Cancel" a second time
 - Temporarily **disable** these **buttons**
 - Saving may fail:
 - **Notify** about this with a **modal window**.
 - Saving was successful
 - **Refresh** the student list.

Changes to <AddStudentForm> 2/2

```
<div className="form-actions">
  <button
    type="button"
    className="btn-cancel"
    onClick={() => navigate(-1)}
    disabled={isSubmitting}
  >
    Cancel
  </button>
  <button
    type="submit"
    className="btn-save"
    disabled={isSubmitting}
  >
    {isSubmitting ? 'Saving...' : 'Save student'}
  </button>
</div>
</form>
</div>
```

- The presence of the disabled attribute is conditional.
 - It depends on the isSubmitting state.
- Similarly, the text on the button depends on this hook.

The effect of the program 1/3

- Left screenshot: attempting to submit a blank form.
- Right screenshot: data just before submission.

Students Courses Time: 20:12:43 Refres

Add a new student

First name:

Last name:

Index no:

Department:

Profile icon:

! Wypełnij to pole.

Add a new student

First name:

Last name:

Index no:

Department:

Profile icon:

The effect of the program 2/3

- Left screenshot - in progress (2 seconds)
- Right screenshot - after another 2 seconds, the student list refreshed

Add a new student

First name:

Last name:

Index no:

Department:

Profile icon:

Academic System

Students

+ Add student

Student list

John Doe (ID: 123456)

Jane Smith (ID: 234567)

Dariusz Konieczny (ID: 555555)

Select a student to view details

The effect of the program 3/3

- After **disabling** the backend.
- After **enabling** it and **adding** another student.
- The **contents** of the `db.json` file change.

Komunikat z witryny localhost:5173: Time: 20:12:43

An error occurred while adding a student!

OK

Add a new student

First name: Failed

Last name: Student

Index no: 222222

Department: Department 2 (W2)

Profile icon: Icon 2

Cancel Saving...

Student list

John Doe (ID: 123456)

Jane Smith (ID: 234567)

Dariusz Konieczny (ID: 555555)

Failed Student (ID: 222222)

```
back-end > {} db.json > ...
1  {
2    "students": [
3      {
4        "id": "1",
5        "index": 123456,
6        "firstName": "John",
7        "lastName": "Doe",
8        "department": "W1",
9        "icon": "/icons/icon1.png"
10     },
11     {
12       "id": "2",
13       "index": 234567,
14       "firstName": "Jane",
15       "lastName": "Smith",
16       "department": "W2",
17       "icon": "/icons/icon2.png"
18     },
19     {
20       "id": 1768677392892,
21       "firstName": "Dariusz",
22       "lastName": "Konieczny",
23       "index": 555555,
24       "department": "W4N",
25       "icon": "/icons/icon4.png"
26     },
27     {
28       "id": 1768677656291,
29       "firstName": "Failed",
30       "lastName": "Student",
31       "index": 222222,
32       "department": "W2",
33       "icon": "/icons/icon2.png"
34     }
35   ]
36 }
```

Styling, spinners

- Many of the styles were generated by Gemini.
- Animated styles (**spinner**) for student loading (left) were also proposed, which create an interesting effect (right).
 - This is a rounded (50%) rectangle (forming a circle), where the top (rounded) edge is a different color.
 - This rectangle rotates from 0 degrees to 360 degrees per second, linearly, indefinitely.

```
front-end > src > features > students > # StudentViewComponent.css >
62  /* loading spinner */
63  .spinner-container {
64    display: flex;
65    flex-direction: column;
66    align-items: center;
67    padding: 20px;
68  }
69
70  .loading-spinner {
71    width: 40px;
72    height: 40px;
73    border: 4px solid #f3f3f3;
74    border-top: 4px solid #3498db;
75    border-radius: 50%;
76    animation: spin 1s linear infinite;
77  }
78
79  @keyframes spin {
80    0% { transform: rotate(0deg); }
81    100% { transform: rotate(360deg); }
82  }
```

